

Data Structures with C

Lecture 5: Asymptotic Analysis

Big-O, Big- Ω and Big- Θ

Dr. Siddhant Thapliyal

August 1, 2026

Why Asymptotic Analysis?

Suppose two algorithms solve the same problem:

- Algorithm A takes 10 operations.
- Algorithm B takes 20 operations.

The difference may not be significant.

However, consider:

- Algorithm A takes 10 operations.
- Algorithm B takes 1000 operations.

Why Asymptotic Analysis?

Suppose two algorithms solve the same problem:

- Algorithm A takes 10 operations.
- Algorithm B takes 20 operations.

The difference may not be significant.

However, consider:

- Algorithm A takes 10 operations.
- Algorithm B takes 1000 operations.

Key Idea

Asymptotic analysis studies how an algorithm grows when the input size n becomes very large.

What Do We Ignore?

Consider:

$$f(n) = 4n^2 + 10n + 50$$

For large values of n :

- $4n^2$ dominates $10n$.
- $4n^2$ dominates the constant 50.
- The constant multiplier 4 is ignored.

Therefore,

$$f(n) = \Theta(n^2)$$

What Do We Ignore?

Consider:

$$f(n) = 4n^2 + 10n + 50$$

For large values of n :

- $4n^2$ dominates $10n$.
- $4n^2$ dominates the constant 50.
- The constant multiplier 4 is ignored.

Therefore,

$$f(n) = \Theta(n^2)$$

Rule

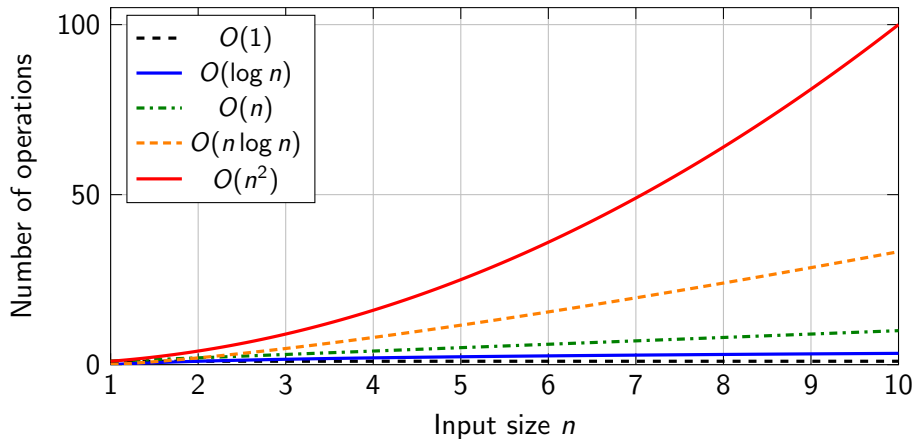
Retain the highest-order term and ignore constants and lower-order terms.

Common Orders of Growth

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

Complexity	Example
$O(1)$	Array element access
$O(\log n)$	Binary search
$O(n)$	Linear search
$O(n \log n)$	Merge sort
$O(n^2)$	Bubble sort
$O(2^n)$	Recursive subset generation

Growth Rate Graph



Understanding the Graph

From the graph:

- $O(1)$ remains constant as n increases.
- $O(\log n)$ grows very slowly.
- $O(n)$ grows directly with input size.
- $O(n \log n)$ grows faster than linear time.
- $O(n^2)$ grows rapidly for large inputs.

Understanding the Graph

From the graph:

- $O(1)$ remains constant as n increases.
- $O(\log n)$ grows very slowly.
- $O(n)$ grows directly with input size.
- $O(n \log n)$ grows faster than linear time.
- $O(n^2)$ grows rapidly for large inputs.

Observation

For small inputs, the difference may appear insignificant. For large inputs, the order of growth becomes critical.

Number of Operations

n	1	$\log_2 n$	n	$n \log_2 n$	n^2
1	1	0	1	0	1
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256
32	1	5	32	160	1024

Number of Operations

n	1	$\log_2 n$	n	$n \log_2 n$	n^2
1	1	0	1	0	1
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256
32	1	5	32	160	1024

Question

Which function grows fastest in the table?

Number of Operations

n	1	$\log_2 n$	n	$n \log_2 n$	n^2
1	1	0	1	0	1
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256
32	1	5	32	160	1024

Question

Which function grows fastest in the table?

$$n^2$$

Big-O Notation: Upper Bound

Definition

A function $f(n)$ is in $O(g(n))$ if there exist positive constants c and n_0 such that

$$0 \leq f(n) \leq c g(n)$$

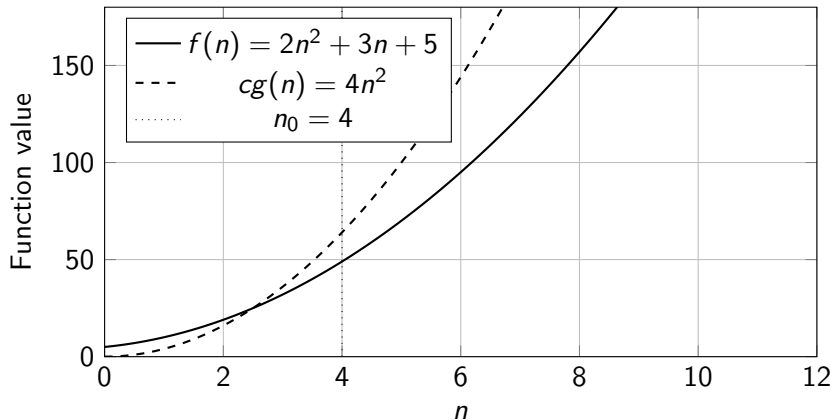
for every $n \geq n_0$.

Big-O provides an asymptotic upper bound.

$$f(n) \in O(g(n))$$

It means that $f(n)$ does not grow faster than a constant multiple of $g(n)$ after some input size n_0 .

Big-O Graphical Interpretation



For $n \geq n_0$, the function $f(n)$ remains below $cg(n)$.

Big-O Example

Show that:

$$4n^2 = O(n^3)$$

According to the definition:

$$4n^2 \leq cn^3$$

Dividing by n^2 :

$$4 \leq cn$$

Choose:

$$c = 4, \quad n_0 = 1$$

Then:

$$4n^2 < 4n^3 \quad \forall n > 1$$

Numerical Problem 1

Determine the Big-O complexity of:

$$f(n) = 7n^3 + 4n^2 + 10n + 25$$

Numerical Problem 1

Determine the Big-O complexity of:

$$f(n) = 7n^3 + 4n^2 + 10n + 25$$

Step 1: Identify the highest-order term.

$$7n^3$$

Numerical Problem 1

Determine the Big-O complexity of:

$$f(n) = 7n^3 + 4n^2 + 10n + 25$$

Step 1: Identify the highest-order term.

$$7n^3$$

Step 2: Ignore the coefficient and lower-order terms.

$$f(n) = O(n^3)$$

Numerical Problem 1

Determine the Big-O complexity of:

$$f(n) = 7n^3 + 4n^2 + 10n + 25$$

Step 1: Identify the highest-order term.

$$7n^3$$

Step 2: Ignore the coefficient and lower-order terms.

$$f(n) = O(n^3)$$

$$\boxed{O(n^3)}$$

Interactive Question 1

Which is the tightest Big-O bound for

$$f(n) = 3n^2 + 20n + 100?$$

- ① $O(1)$
- ② $O(n)$
- ③ $O(n^2)$
- ④ $O(n^3)$

Interactive Question 1

Which is the tightest Big-O bound for

$$f(n) = 3n^2 + 20n + 100?$$

- ① $O(1)$
- ② $O(n)$
- ③ $O(n^2)$
- ④ $O(n^3)$

Answer

The tightest upper bound is

$$O(n^2)$$

because n^2 is the dominant term.

Big-Ω Notation: Lower Bound

Definition

A function $f(n)$ is in $\Omega(g(n))$ if there exist positive constants c and n_0 such that

$$0 \leq c g(n) \leq f(n)$$

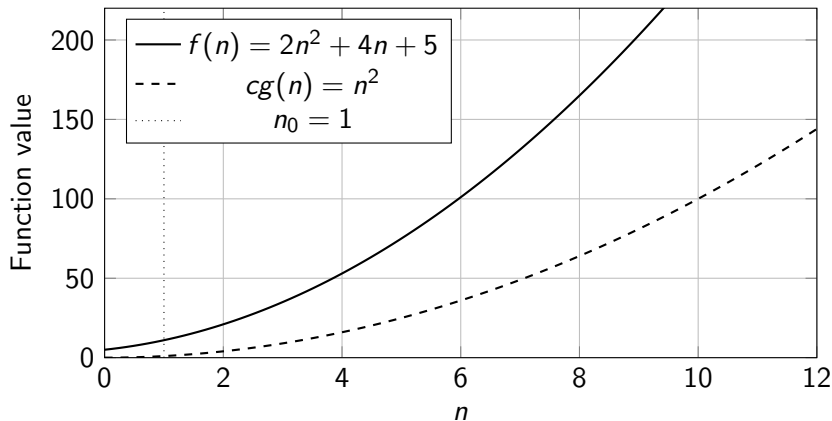
for every $n \geq n_0$.

Big-Ω provides an asymptotic lower bound.

$$f(n) \in \Omega(g(n))$$

It guarantees that $f(n)$ grows at least as fast as a constant multiple of $g(n)$.

Big-Ω Graphical Interpretation



For $n \geq n_0$, $f(n)$ remains above $cg(n)$.

Big- Ω Example

Show that:

$$5n^2 + 10n = \Omega(n^2)$$

According to the definition:

$$cn^2 \leq 5n^2 + 10n$$

Choose:

$$c = 5$$

Then:

$$5n^2 \leq 5n^2 + 10n$$

This is true for every $n \geq 1$.

Therefore,

Interactive Question 2

Is the following statement true?

$$7n = \Omega(n^2)$$

- 1 True
- 2 False

Interactive Question 2

Is the following statement true?

$$7n = \Omega(n^2)$$

- 1 True
- 2 False

For the statement to be true:

$$cn^2 \leq 7n$$

Dividing by n :

$$cn \leq 7$$

This cannot hold for every sufficiently large n using a fixed positive constant c .

Interactive Question 2

Is the following statement true?

$$7n = \Omega(n^2)$$

- 1 True
- 2 False

For the statement to be true:

$$cn^2 \leq 7n$$

Dividing by n :

$$cn \leq 7$$

This cannot hold for every sufficiently large n using a fixed positive constant c .

False

Big- Θ Notation: Tight Bound

Definition

A function $f(n)$ is in $\Theta(g(n))$ if there exist positive constants c_1 , c_2 and n_0 such that

$$0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$$

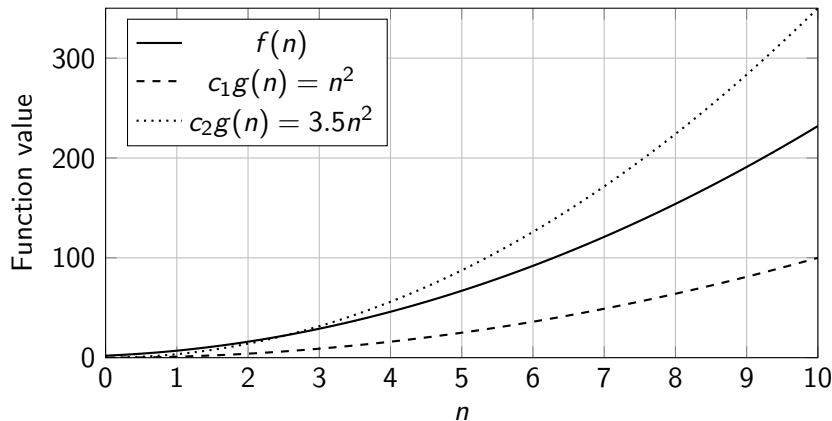
for every $n \geq n_0$.

Big- Θ provides an asymptotically tight bound.

$$f(n) \in \Theta(g(n))$$

It gives both an upper bound and a lower bound.

Big- Θ Graphical Interpretation



For sufficiently large n , $f(n)$ lies between two constant multiples of $g(n)$.

Theta Example

Show that:

$$3n^2 + 5n + 2 = \Theta(n^2)$$

For $n \geq 1$:

$$3n^2 \leq 3n^2 + 5n + 2$$

Also, because $n \leq n^2$ and $1 \leq n^2$:

$$3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$$

Therefore:

$$3n^2 \leq 3n^2 + 5n + 2 \leq 10n^2$$

Choose:

$$c_1 = 3 \quad c_2 = 10 \quad n_0 = 1$$

Relationship Between the Notations

$$f(n) = \Theta(g(n))$$

if and only if:

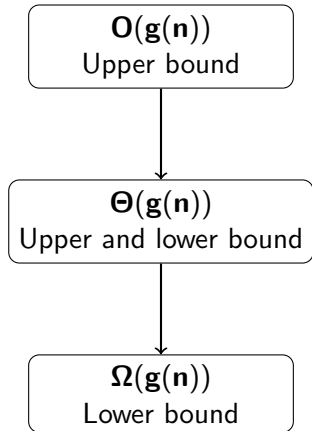
$$f(n) = O(g(n))$$

and

$$f(n) = \Omega(g(n))$$

Notation	Bound	Interpretation
$O(g(n))$	Upper	At most this growth rate
$\Omega(g(n))$	Lower	At least this growth rate
$\Theta(g(n))$	Tight	Same asymptotic growth rate

Visual Summary



Example: Linear Loop

```
for (i = 0; i < n; i++)  
{  
    printf("%d", i);  
}
```

The statement executes n times.

$$T(n) = an + b$$

Therefore:

$$T(n) = \Theta(n)$$

It is also correct that:

$$T(n) = O(n) \quad \text{and} \quad T(n) = \Omega(n)$$

Example: Logarithmic Loop

```
for (i = 1; i < n; i *= 2)
{
    printf("%d", i);
}
```

After k iterations:

$$i = 2^k$$

The loop stops when:

$$2^k \geq n$$

Therefore:

$$k = \log_2 n$$

Hence:

Example: Quadratic Loop

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        printf("*");  
    }  
}
```

The outer loop runs n times.

The inner loop runs n times for each outer iteration.

$$T(n) = n \times n = n^2$$

Therefore:

$$\Theta(n^2)$$

Example: Dependent Nested Loop

```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j <= i; j++)  
    {  
        printf("*");  
    }  
}
```

Total iterations:

$$1 + 2 + 3 + \dots + n$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = \frac{n^2 + n}{2}$$

Interactive Question 3

Determine the complexity:

```
for (i = 1; i <= n; i++)  
{  
    for (j = 1; j <= n; j *= 2)  
    {  
        printf("*");  
    }  
}
```

- 1 $\Theta(n)$
- 2 $\Theta(\log n)$
- 3 $\Theta(n \log n)$
- 4 $\Theta(n^2)$

Interactive Question 3

Determine the complexity:

```
for (i = 1; i <= n; i++)  
{  
    for (j = 1; j <= n; j *= 2)  
    {  
        printf("*");  
    }  
}
```

- ① $\Theta(n)$
- ② $\Theta(\log n)$
- ③ $\Theta(n \log n)$
- ④ $\Theta(n^2)$

Outer loop:

$\Theta(n)$

Interactive Question 3

Determine the complexity:

```
for (i = 1; i <= n; i++)  
{  
    for (j = 1; j <= n; j *= 2)  
    {  
        printf("*");  
    }  
}
```

- ① $\Theta(n)$
- ② $\Theta(\log n)$
- ③ $\Theta(n \log n)$
- ④ $\Theta(n^2)$

Outer loop:

$\Theta(n)$

Interactive Question 4

Determine the complexity:

```
for (i = n; i > 1; i /= 2)
{
    for (j = 0; j < n; j++)
    {
        printf("*");
    }
}
```

Interactive Question 4

Determine the complexity:

```
for (i = n; i > 1; i /= 2)
{
    for (j = 0; j < n; j++)
    {
        printf("*");
    }
}
```

Outer loop:

$$\Theta(\log n)$$

Inner loop:

$$\Theta(n)$$

Interactive Question 4

Determine the complexity:

```
for (i = n; i > 1; i /= 2)
{
    for (j = 0; j < n; j++)
    {
        printf("*");
    }
}
```

Outer loop:

$$\Theta(\log n)$$

Inner loop:

$$\Theta(n)$$

Therefore:

Numerical Problem 2

Find the tight asymptotic bound:

$$f(n) = 20n^4 + 4n^3 + 100n + 500$$

Numerical Problem 2

Find the tight asymptotic bound:

$$f(n) = 20n^4 + 4n^3 + 100n + 500$$

The highest-order term is:

$$20n^4$$

Ignoring the constant multiplier:

$$f(n) = \Theta(n^4)$$

Numerical Problem 3

Arrange the following functions in increasing order of growth:

$$n^2, \log n, 2^n, n, n \log n, 1$$

Numerical Problem 3

Arrange the following functions in increasing order of growth:

$$n^2, \quad \log n, \quad 2^n, \quad n, \quad n \log n, \quad 1$$

$$1 < \log n < n < n \log n < n^2 < 2^n$$

Numerical Problem 4

Is the following statement correct?

$$n^2 = O(n^3)$$

Numerical Problem 4

Is the following statement correct?

$$n^2 = O(n^3)$$

We require:

$$n^2 \leq cn^3$$

Dividing by n^2 :

$$1 \leq cn$$

Choose:

$$c = 1, \quad n_0 = 1$$

Therefore:

$$n^2 = O(n^3)$$

Numerical Problem 5

Is the following statement correct?

$$n^3 = O(n^2)$$

Numerical Problem 5

Is the following statement correct?

$$n^3 = O(n^2)$$

We would require:

$$n^3 \leq cn^2$$

Dividing by n^2 :

$$n \leq c$$

No fixed positive constant c can satisfy this for every sufficiently large n .
Therefore:

$$n^3 \neq O(n^2)$$

Rapid-Fire Questions

Determine the tight bound for each function:

- ① $8n + 20$
- ② $4n^2 + 3n + 1$
- ③ $n \log n + 10n$
- ④ 5
- ⑤ $2^n + n^5$

Rapid-Fire Questions

Determine the tight bound for each function:

① $8n + 20$

② $4n^2 + 3n + 1$

③ $n \log n + 10n$

④ 5

⑤ $2^n + n^5$

① $\Theta(n)$

② $\Theta(n^2)$

③ $\Theta(n \log n)$

④ $\Theta(1)$

⑤ $\Theta(2^n)$

True or False

- ① $5n = O(n^2)$
- ② $n^2 = \Omega(n)$
- ③ $3n^2 + 4n = \Theta(n^2)$
- ④ $2^n = O(n^3)$
- ⑤ $n \log n = \Omega(n)$

True or False

- ① $5n = O(n^2)$
- ② $n^2 = \Omega(n)$
- ③ $3n^2 + 4n = \Theta(n^2)$
- ④ $2^n = O(n^3)$
- ⑤ $n \log n = \Omega(n)$

- ① True
- ② True
- ③ True
- ④ False
- ⑤ True

Think–Pair–Share Activity

Consider two algorithms:

$$T_A(n) = 100n$$

$$T_B(n) = n^2$$

Questions

- 1 Which algorithm has the better asymptotic complexity?
- 2 Which algorithm is faster when $n = 10$?
- 3 At approximately what input size does Algorithm A become faster?

Think–Pair–Share Activity

Consider two algorithms:

$$T_A(n) = 100n$$

$$T_B(n) = n^2$$

Questions

- 1 Which algorithm has the better asymptotic complexity?
- 2 Which algorithm is faster when $n = 10$?
- 3 At approximately what input size does Algorithm A become faster?

For the crossover point:

$$100n = n^2$$

Common Mistakes

- Treating Big-O as an exact running time.
- Including constant multipliers in the final answer.
- Keeping lower-order terms.
- Confusing upper bounds with tight bounds.
- Assuming Big-O always means the best case or worst case.
- Multiplying the complexity of sequential loops.
- Adding the complexity of nested loops.

Common Mistakes

- Treating Big-O as an exact running time.
- Including constant multipliers in the final answer.
- Keeping lower-order terms.
- Confusing upper bounds with tight bounds.
- Assuming Big-O always means the best case or worst case.
- Multiplying the complexity of sequential loops.
- Adding the complexity of nested loops.

Remember

Sequential operations are generally added; nested operations are generally multiplied.

Big-O versus Case Analysis

Asymptotic notation and case analysis describe different ideas.

- Best, average and worst case describe different input situations.
- O , Ω and Θ describe mathematical growth bounds.

For linear search:

$$T_{\text{best}}(n) = \Theta(1)$$

$$T_{\text{average}}(n) = \Theta(n)$$

$$T_{\text{worst}}(n) = \Theta(n)$$

Important

Big-O should not automatically be treated as “worst case.” A best-case, average-case or worst-case function can each be expressed using asymptotic notation.

Tutorial Problems

Solve the following:

- 1 Show that $6n^2 + 2n + 4 = O(n^2)$.
- 2 Show that $4n^3 + 10n = \Omega(n^3)$.
- 3 Prove that $7n^2 + 5n + 1 = \Theta(n^2)$.
- 4 Determine the complexity of three nested loops, each running n times.
- 5 Determine the complexity of a loop in which the variable is divided by 3.
- 6 Arrange $\sqrt{n}$, n , $\log n$, n^2 and $n \log n$ in increasing order.

Before leaving, answer these questions:

- 1 What is the difference between $O(g(n))$ and $\Theta(g(n))$?
- 2 What is the dominant term of $5n^3 + 20n^2 + 100$?
- 3 What is the complexity of a loop in which i doubles?
- 4 Is $n^2 = O(n^4)$ correct? Is it a tight bound?

Before leaving, answer these questions:

- 1 What is the difference between $O(g(n))$ and $\Theta(g(n))$?
- 2 What is the dominant term of $5n^3 + 20n^2 + 100$?
- 3 What is the complexity of a loop in which i doubles?
- 4 Is $n^2 = O(n^4)$ correct? Is it a tight bound?

Discuss your answers with a partner.

Questions?

Thank You